

CASE STUDY REPORT

Limitations of Low-Code Platforms for a Complex Enterprise Use Case

Experiment 13

Name: MAITREYEE DD

Register No.: 241401050

Table of Contents

1. Summary
2. What Is Low-Code?
3. The Case: A Hospital Network Goes Low-Code
4. Where It Went Wrong
5. What It Cost
6. Low-Code vs Normal Coding
7. What Should Have Been Done
8. When Low-Code Is Still a Good Idea
9. Conclusion
10. References

1. Summary

Low-code platforms let people build apps by dragging blocks on a screen instead of writing every line of code. They are great for small tools, forms and approval flows. But what happens when a company uses one for a huge, critical system? This case study looks at a made-up hospital network that tried exactly that. The first few screens were built very fast, but the hard parts (speed, complicated rules, old systems, rising fees) caused delays and extra cost. The report lists these problems in simple terms and suggests a safer approach: use low-code for the easy outer layer and normal code for the heavy core.

2. What Is Low-Code?

A low-code platform is a ready-made workshop for building apps. It gives you drag-and-drop screens, ready-made login, built-in workflow tools and one-click publishing. You write only small bits of code when needed. Examples are OutSystems, Mendix, Microsoft Power Apps and Zoho Creator.

The good side is speed and lower effort. The catch is that the platform decides how your app runs behind the scenes. You get less control over the database, the speed and the servers. For a simple app this does not matter. For a very big app, it matters a lot.

3. The Case: A Hospital Network Goes Low-Code

3.1 The Company

CareBridge Health is an imaginary hospital group with 40 hospitals and 150 clinics. (The name and numbers are made up for this report.) Its old patient system was 15 years old, slow, and hard to change. The management picked a popular low-code platform to replace it in 12 months, hoping to finish faster and cheaper than a normal rewrite.

3.2 Why This System Is Complex

- **Heavy use:** Around 2 lakh patient visits a day across all branches.
- **Many rules:** Thousands of billing and insurance rules that differ for each insurer and change often.
- **Many connections:** Lab machines, pharmacy stock, scanning devices, insurance portals and the government health portal all had to be linked.
- **Strict privacy:** Patient records are very private and must follow strict laws and audits.
- **Always on:** The system must run 24 hours a day for 10+ years with no long breaks.

3.3 What Happened

In the first two months the team built patient registration and appointment booking. Doctors and staff loved it. Everyone believed the project would finish early. Then the team reached insurance billing, lab-machine links and the busy-hour patient search. Progress slowed down badly. To get past the platform's limits, the team wrote a lot of extra code and built separate services outside the platform. In the end the project was changed to a mixed design, where low-code handled only the screens and simple workflows. It finished late and cost more than a normal build would have.

4. Where It Went Wrong

4.1 Slow When Many People Use It

In morning rush hours, thousands of staff searched and updated records together. The platform became slow because the team could not tune the database or choose how data was stored. Fixing speed meant raising a request with the vendor or paying for a bigger plan.

4.2 Hard to Handle Complicated Rules

Insurance and billing rules have many "if this, then that" branches. On a visual screen these turned into giant diagrams that nobody could read or check. Testing them was hard, and one small change could break another rule without anyone noticing.

4.3 Trouble Connecting Old Systems

Ready-made connectors worked for common tools like email. But old lab machines and insurer portals used unusual formats. The team had to write special adapters, and these broke whenever the platform released an update.

4.4 Locked to One Vendor

The app lived in the vendor's own format. If CareBridge wanted to leave, almost everything would need to be rebuilt. This gave the vendor power during price talks.

4.5 Limited Freedom

Some needs, like a special queue-management logic for emergency patients, did not fit into the platform's ready-made blocks. Every workaround added mess that someone must clean later.

4.6 Privacy and Audit Worries

Hospital data is highly sensitive. Auditors asked where the data was stored and how it was protected. CareBridge could only show the vendor's documents, not check things itself. Some rules needed a costly private hosting option.

4.7 Too Many Small Apps

Because building apps was easy, different departments created their own apps. Soon there were duplicate forms, different security settings and nobody clearly in charge.

4.8 Cost Kept Growing

Fees depend on users, apps and data. During the trial the bill was small. After adding thousands of staff and more test environments, it grew fast. Add expert contractors and workaround coding, and the promised saving disappeared.

4.9 Skills Gap

Experts in that exact platform were rare and expensive. Connecting the platform to normal tools such as version control and automatic testing was also limited. The belief that low-code means no developers turned out to be wrong.

Problem	What was seen	Seriousness
Speed at scale	Slow at peak hours	High

Problem	What was seen	Seriousness
Complex rules	Unreadable, hard to test	High
Old system links	Fragile custom adapters	High
Vendor lock-in	Costly to leave	High
Limited freedom	Workarounds piling up	Medium
Privacy / audit	Depends on vendor proof	High
Too many apps	Duplicates, no owner	Medium
Cost	Fees grew past plan	High
Skills	Rare, costly experts	Medium

Table 1: Summary of problems

5. What It Cost

The table compares the plan with what really happened. These numbers are made up to show the idea.

Item	Plan	Actual (example)
Time to finish	12 months	19 months
Cost compared with normal build	30% cheaper	About 15% costlier
Share built with low-code	100%	About 35%
Daily visits handled smoothly	2 lakh	About 1 lakh at first
Extra custom code	Very little	A large amount

Table 2: Plan versus reality (illustrative)

Besides money and time, the hospital faced worried doctors, delayed insurance claims and a tired development team.

6. Low-Code vs Normal Coding

Point	Low-Code	Normal Coding
Starting speed	Very fast	Slower
Complicated logic	Hard to manage	Easy to control and test
Speed tuning	Limited by vendor	Full control
Linking old systems	Only common ones are easy	Anything is possible
Moving away later	Very hard	Easier
Cost over time	Low first, grows later	High first, steadier
Best for	Forms, workflows, prototypes	Core and heavy systems

Table 3: Comparison

7. What Should Have Been Done

- **Test the difficult part early.** Try the hardest part first (billing rules, peak load) in a small test before signing a big deal.
- **Use a mixed design.** Use low-code for screens and simple workflows. Keep billing, rules and data-heavy work in normal code, linked by APIs.
- **Plan the exit.** Keep business rules written down outside the platform and ask for data export rights in the contract.
- **Check the total cost.** Count the full price for 5 to 10 years, including extra environments and experts.
- **Control the apps.** Set naming rules, security rules and an owner for every app from day one.
- **Collect proof for audits.** Ask the vendor for certificates, test reports and data-location proof before finalising the design.

8. When Low-Code Is Still a Good Idea

This does not mean low-code is bad. It works very well for leave-approval apps, feedback forms, dashboards, appointment reminders, small internal tools and quick prototypes. The mistake in this case was not choosing low-code. The mistake was expecting it to carry the hospital's biggest and most critical workload.

9. Conclusion

Low-code gives a great start, but a fast start does not always mean a good finish. For large, complicated, heavily regulated systems, limits in speed, control, cost and freedom can cancel the early gains. The best path is a balanced one: low-code where it is strong, normal coding where control is needed, and clear planning for governance and exit. Low-code should be treated as one useful tool, not a replacement for good software engineering.

10. References

- Gartner. Research on enterprise low-code application platforms.
- Forrester. Reports on low-code development platforms.
- Bock, A. C. and Frank, U. (2021). Low-Code Platform. Business & Information Systems Engineering, 63(6).
- Official documentation of OutSystems, Mendix, Microsoft Power Apps and Zoho Creator.
- Note: CareBridge Health and all figures are fictional and used only for learning.